浙江大学
ZHEJIANG UNIVERSITY




# 人工智能实验

## 实验报告

| | |
|---|---|
| 实验名称 | 实验 1-1 分类模型 |
| 实验地点 | 玉泉高分子楼-203 |
| 姓　名 | 郑智瑄 |
| 学　号 | 3230103913 |
| 实验日期 | March 6, 2026 |
| 指导老师 | 魏准 |

# Contents


**1 实验目的** **1**

**2 实验任务** **1**

**3 实验设备** **1**

**4 提供的代码** **1**

**5 实验内容** **1**

**6 实验过程与结果** **2**

6.1 **4 位先行进位加法器** . . . . . 2

6.1.1 设计原理 . . . . . 2

6.1.2 **Verilog HDL** 代码与设计分析 . . . . . 2

6.1.3 **ModelSim** 仿真与结果分析 . . . . . 5

6.2 **32 位进位选择加法器** . . . . . 5

6.2.1 设计原理 . . . . . 5

6.2.2 **Verilog HDL** 代码与设计分析 . . . . . 6

6.2.3 **ModelSim** 仿真与结果分析 . . . . . 9

6.2.4 时序仿真 . . . . . 9

6.3 **32 位流水线加法器** . . . . . 9

6.3.1 设计原理 . . . . . 9

6.3.2 **Verilog HDL** 代码与设计分析 . . . . . 10

6.3.3 **ModelSim** 仿真与结果分析 . . . . . 13

6.3.4 时序仿真与结果分析 . . . . . 13

**7 遇到的问题及解决方法** **13**

**8 思考题** **14**

# 1 实验目的

1. 掌握快速加法器的设计方法。
2. 熟悉流水线技术。
3. 掌握时序仿真的工作流程

# 2 实验任务

1. 采用" 进位选择加法" 技术设计 32 位加法器，并对设计进行功能仿真和时序仿真。
2. 采用四级流水线技术设计 32 位加法器，并对设计进行功能仿真和时序仿真。

# 3 实验设备

装有 Vivado、ModelSim SE 软件的计算机。

# 4 提供的代码

1. 4 位加法器测试代码 adder_4bits_tb.v。
2. 32 位进位选择加法器测试代码 adder_32bits_tb.v。
3. 32 位流水线加法器测试代码 pipeline_adder_tb.v。

# 5 实验内容

1. 编写 4 位先行进位加法器的 Verilog HDL 代码，并用 ModelSim 软件进行功能仿真。
2. 32 位进位选择加法器的设计
   (1) 编写用 32 位进位选择加法器的 Verilog HDL 代码，并用 ModelSim 软件进行功能。
   (2) 对 32 位加法器进行时序仿真
3. 编写采用 4 级流水线技术的 32 位加法器的 Verilog HDL 代码，并对设计进行时序仿真。

# 6 实验过程与结果

## 6.1 4 位先行进位加法器

### 6.1.1 设计原理

两个加数分别为 $A_3A_2A_1A_0$ 和 $B_3B_2B_1B_0$，$C_{-1}$ 为最低位进位。设两个辅助变量分别为 $G_3G_2G_1G_0$ 和 $P_3P_2P_1P_0$；$G_i = A_i\&B_i$、$P_i = A_i + B_i$。

一位全加器的逻辑表达式可转化为

$$\begin{cases} S_i = P_i\overline{G_i} \oplus C_{i-1} \\ C_i = G_i + P_iC_{i-1} \end{cases} \tag{1}$$

利用上述关系，一个四位加法器的进位计算就变转化为

$$\begin{cases} C_0 = G_0 + P_0C_{-1} \\ C_1 = G_1 + P_1C_0 = G_1 + P_1G_0 + P_1P_0C_{-1} \\ C_2 = G_2 + P_2C_1 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_{-1} \\ C_3 = G_3 + P_3C_2 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0C_{-1} \end{cases} \tag{2}$$

由式（5.13）可以看出，每一个进位的计算都直接依赖于整个加法器的最初输入，而不需要等待相邻低位的进位传递。理论上，每一个进位的计算都只需要三个门延迟时间，即产生 $G[i]$、$P[i]$ 的与门和或门，输入为 $G[i]$、$P[i]$、$C_{-1}$ 的与门，以及最终的或门。同样道理，理论上最终结果 sum 的得到只需要四个门延迟时间。

实际上，当加数位数较大时，输入需要驱动的门数量较多，其 VLSI 实现的输出时延增加很多，考虑到互连线的延时情况将会更加糟糕。因此，通常在芯片实现时先设计位数较少的超前进位加法器结构，而后以此为基本结构来构造位数较大的加法器。

### 6.1.2 Verilog HDL 代码与设计分析

1. **模块接口：**

   - a,b:4 位输入信号，两个需要相加的 4 位二进制数。
   - ci(carry in): 进位输入，低位加法器的进位。
   - s(sum):4 位的和输出。
   - co(carry out): 进位输出，最高位的进位。

2. **内部信号：**

- g(generate): 生成进位
- p(propagate): 传播进位
- c(carry): 内部进位，存储每个位的进位值。

3. **部分电路图 (注：标注与实验原理中有所不同)：**

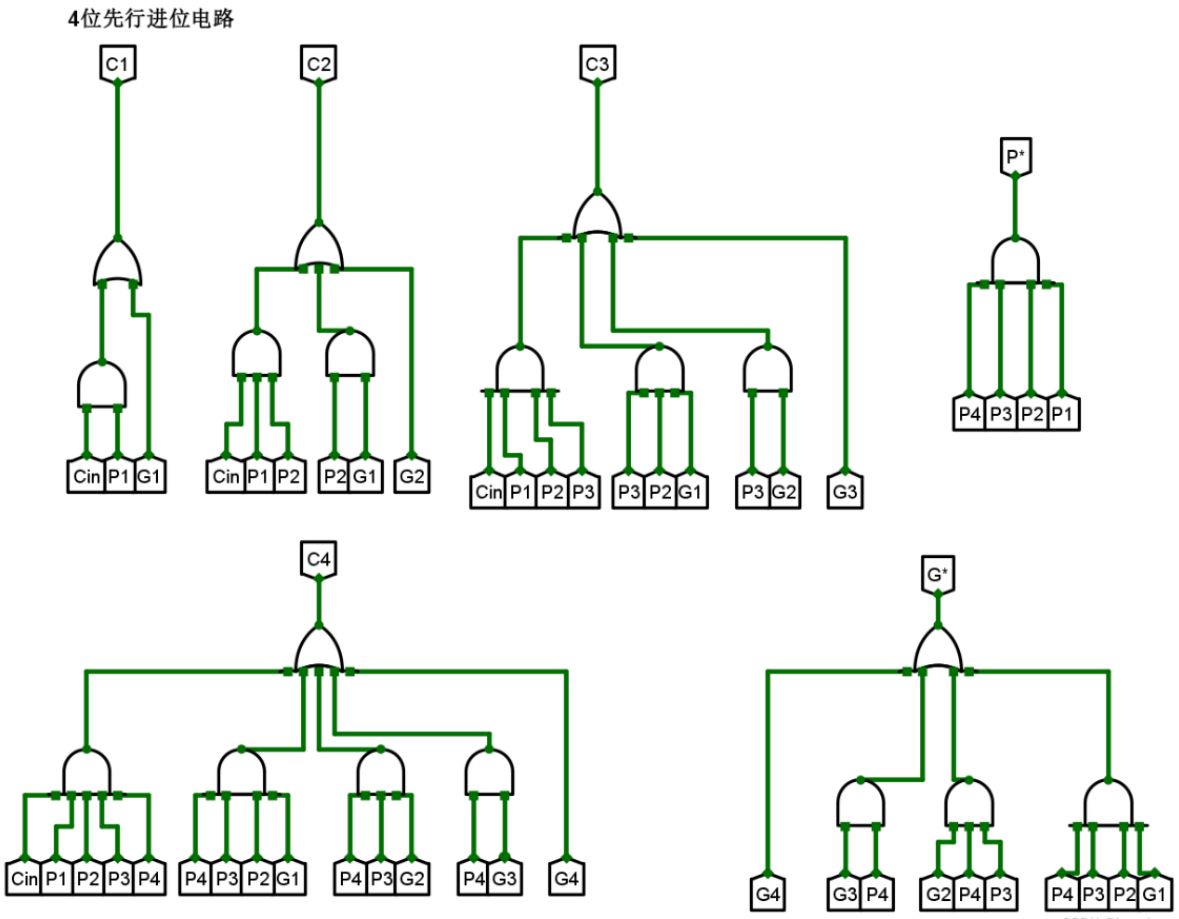



Figure 1: 4 位先行进位加法器 ModelSim 仿真结果

Listing 1: 4 位先行进位加法器 Verilog HDL 代码

```
module adder_4bits(
    input [3:0] a,
    input [3:0] b,
    input ci,
    output [3:0] s,
    output co
);

    wire [3:0] g; // 进位生成信号 (Generate)
    wire [3:0] p; // 进位传播信号 (Propagate)
    wire [2:0] c; // 中间进位信号

    // 计算进位生成和传播信号
    assign g = a & b;
    assign p = a | b;

    // 超前进位逻辑
    assign c[0] = g[0] | (p[0] & ci);
    assign c[1] = g[1] | (p[1] & g[0]) | (p[1] & p[0] & ci);
    assign c[2] = g[2] | (p[2] & g[1]) | (p[2] & p[1] & g[0]) | (p[2] & p
        [1] & p[0] & ci);
    assign co = g[3] | (p[3] & g[2]) | (p[3] & p[2] & g[1]) | (p[3] & p[2]
         & p[1] & g[0]) | (p[3] & p[2] & p[1] & p[0] & ci);

    // 计算和
    assign s[0] = (p[0] & ~g[0]) ^ ci;
    assign s[1] = (p[1] & ~g[1]) ^ c[0];
    assign s[2] = (p[2] & ~g[2]) ^ c[1];
    assign s[3] = (p[3] & ~g[3]) ^ c[2];

endmodule
```

### 6.1.3 ModelSim 仿真与结果分析

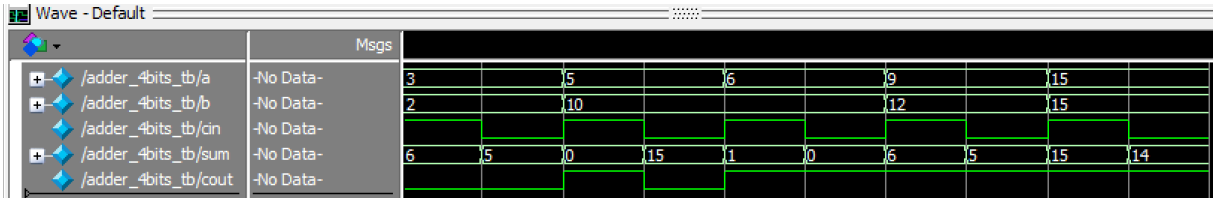



Figure 2: 4 位先行进位加法器 ModelSim 仿真结果

【实验结果】: 仿真结果表示，4 位先行进位加法器功能正常。

## 6.2 32 位进位选择加法器

### 6.2.1 设计原理

实际上，由超前进位加法器级联构成的多位加法器只是提高了进位传递的速度，其计算过程与行波进位加法器同样需要等待进位传递的完成。

借鉴并行计算的思想，人们提出了进位选择加法器结构，或者称为有条件的加法器结构（conditional sum adder），其算法的实质是增加硬件面积换取速度性能的提高。二进制加法的特点是进位或者为逻辑 1，或者为逻辑 0，二者必居其一。将进位链较长的加法器分为 $M$ 块，分别进行加法计算，对除去最低位计算块外的 $M-1$ 块加法结构复制两份，其进位输入分别预定为逻辑 1 和逻辑 0。于是，$M$ 块加法器可以同时并行进行各自的加法运算，然后根据各自相邻低位加法运算结果产生的进位输出，选择正确的加法结果输出。图 5.6 所示为 12 位进位选择加法器的逻辑结构图。12 位加法器划分为 3 块，最低一块（4 位）由 4 位超前进位加法器直接构成，后两块分别假设前一块的进位为 0 或 1 将两种结果都计算出来，再根据前级进位选择正确的和与进位。如果每一块加法结构内部都采用速度较快的超前进位加法器结构，那么进位选择加法器的计算时延为

$$t_{CSA} = t_{carry} + (M-2)t_{MUX} + t_{sum} \tag{3}$$

其中，$t_{sum}$、$t_{carry}$ 分别为加法器的和与加法器的进位时延，$t_{MUX}$ 为数据选择器的时延。

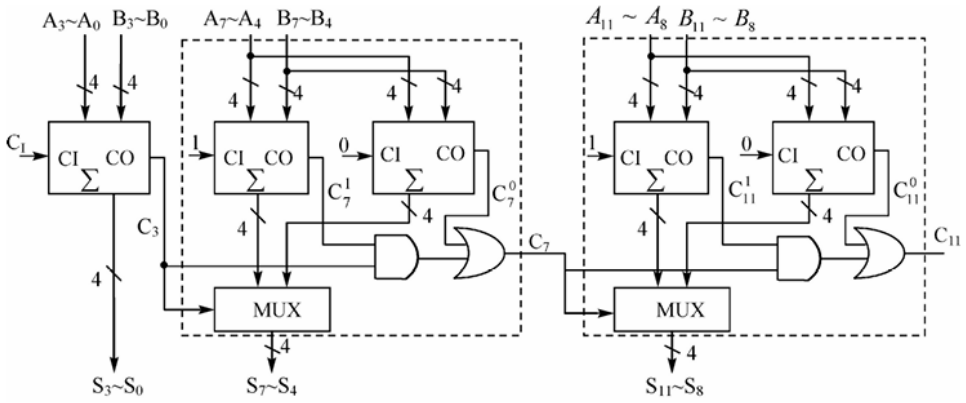



Figure 3: 12 位进位选择加法器原理图

### 6.2.2 Verilog HDL 代码与设计分析

1. **模块接口：**

   - a,b:32 位输入信号，表示两个需要相加的 32 位二进制数。
   - ci(carry in): 进位输入，低位加法器的进位。
   - s(sum):32 位的和输出，表示加法结果。
   - co(carry out): 进位输出，表示最高位的进位。

2. **内部信号：**

   - c1∼c7: 各 4 位段之间的进位信号，用于连接 8 个 4 位加法器模块。
   - s0_0, s0_1, . . ., s7_0, s7_1: 每个 4 位段在进位为 0 和进位为 1 时的两种可能的和结果。
   - co0_0, co0_1, . . ., co7_0, co7_1: 每个 4 位段在进位为 0 和进位为 1 时的两种可能的进位输出。

3. **设计思路：**

   - 将 32 位加法器分为 8 个 4 位段，每段使用先行进位加法器实现。
   - 第 0 段直接使用输入进位 ci 进行计算。
   - 第 1∼7 段分别预计算进位为 0 和进位为 1 的两种结果，然后根据前一段的实际进位输出，通过二选一多路选择器选择正确的结果。
   - 这种设计使得各段可以并行计算，大大减少了进位传播延迟，提高了运算速度。

Listing 2: 32 位进位选择加法器 Verilog HDL 代码

```
module adder_32bits(
    input [31:0] a,
    input [31:0] b,
    input ci,
    output [31:0] s,
    output co
);

    wire c1, c2, c3, c4, c5, c6, c7; // 各4位段之间的进位信号

    // 每个4位段的两种可能结果(进位为0和进位为1时)
    wire [3:0] s1_0, s1_1, s2_0, s2_1, s3_0, s3_1, s4_0, s4_1;
    wire [3:0] s5_0, s5_1, s6_0, s6_1, s7_0, s7_1;
    wire       co1_0, co1_1, co2_0, co2_1, co3_0, co3_1, co4_0, co4_1;
    wire       co5_0, co5_1, co6_0, co6_1, co7_0, co7_1;

    // 第0段(bit[3:0]): 直接使用实际进位ci
    adder_4bits u0 (.a(a[3:0]), .b(b[3:0]), .ci(ci), .s(s[3:0]), .co(c1));

    // 第1段(bit[7:4]): 预计算进位为0和1的两种结果，根据c1选择
    adder_4bits u1_0 (.a(a[7:4]), .b(b[7:4]), .ci(1'b0), .s(s1_0), .co(
        co1_0));
    adder_4bits u1_1 (.a(a[7:4]), .b(b[7:4]), .ci(1'b1), .s(s1_1), .co(
        co1_1));
    assign s[7:4] = c1 ? s1_1 : s1_0;
    assign c2   = c1 ? co1_1 : co1_0;

    // 第2段(bit[11:8]): 根据c2选择结果
    adder_4bits u2_0 (.a(a[11:8]), .b(b[11:8]), .ci(1'b0), .s(s2_0), .co(
        co2_0));
    adder_4bits u2_1 (.a(a[11:8]), .b(b[11:8]), .ci(1'b1), .s(s2_1), .co(
        co2_1));
    assign s[11:8] = c2 ? s2_1 : s2_0;
    assign c3    = c2 ? co2_1 : co2_0;

    // 第3段(bit[15:12]): 根据c3选择结果
    adder_4bits u3_0 (.a(a[15:12]), .b(b[15:12]), .ci(1'b0), .s(s3_0), .co
```

```
    (co3_0));
34  adder_4bits u3_1 (.a(a[15:12]), .b(b[15:12]), .ci(1'b1), .s(s3_1), .co
    (co3_1));
35  assign s[15:12] = c3 ? s3_1 : s3_0;
36  assign c4     = c3 ? co3_1 : co3_0;
37
38  // 第4段(bit[19:16]): 根据c4选择结果
39  adder_4bits u4_0 (.a(a[19:16]), .b(b[19:16]), .ci(1'b0), .s(s4_0), .co
    (co4_0));
40  adder_4bits u4_1 (.a(a[19:16]), .b(b[19:16]), .ci(1'b1), .s(s4_1), .co
    (co4_1));
41  assign s[19:16] = c4 ? s4_1 : s4_0;
42  assign c5     = c4 ? co4_1 : co4_0;
43
44  // 第5段(bit[23:20]): 根据c5选择结果
45  adder_4bits u5_0 (.a(a[23:20]), .b(b[23:20]), .ci(1'b0), .s(s5_0), .co
    (co5_0));
46  adder_4bits u5_1 (.a(a[23:20]), .b(b[23:20]), .ci(1'b1), .s(s5_1), .co
    (co5_1));
47  assign s[23:20] = c5 ? s5_1 : s5_0;
48  assign c6     = c5 ? co5_1 : co5_0;
49
50  // 第6段(bit[27:24]): 根据c6选择结果
51  adder_4bits u6_0 (.a(a[27:24]), .b(b[27:24]), .ci(1'b0), .s(s6_0), .co
    (co6_0));
52  adder_4bits u6_1 (.a(a[27:24]), .b(b[27:24]), .ci(1'b1), .s(s6_1), .co
    (co6_1));
53  assign s[27:24] = c6 ? s6_1 : s6_0;
54  assign c7     = c6 ? co6_1 : co6_0;
55
56  // 第7段(bit[31:28]): 根据c7选择结果并输出最终进位
57  adder_4bits u7_0 (.a(a[31:28]), .b(b[31:28]), .ci(1'b0), .s(s7_0), .co
    (co7_0));
58  adder_4bits u7_1 (.a(a[31:28]), .b(b[31:28]), .ci(1'b1), .s(s7_1), .co
    (co7_1));
59  assign s[31:28] = c7 ? s7_1 : s7_0;
60  assign co     = c7 ? co7_1 : co7_0;
61
```

```
62 endmodule
```

### 6.2.3 ModelSim 仿真与结果分析

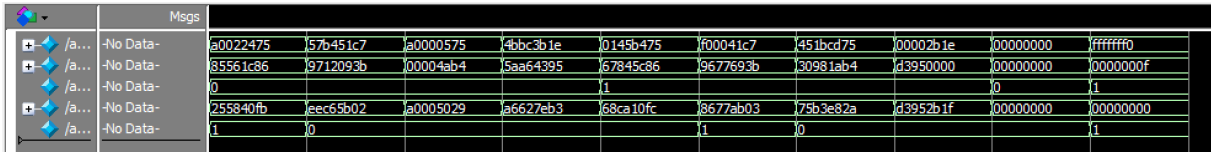



Figure 4: 32 位进位选择加法器 ModelSim 仿真结果

【实验结果】:ModelSim 仿真结果表示，32 位进位选择加法器功能正常。

### 6.2.4 时序仿真

在 vivado 中进行综合和实现后，对 32 位进位选择加法器进行时序仿真，仿真结果如下，可以看到时序仿真中信号存在建立时间和毛刺等现象

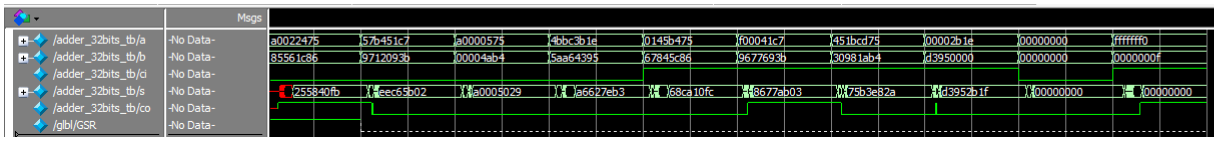



Figure 5: 32 位进位选择加法器时序仿真结果

【实验结果】: 时序仿真结果表示，32 位进位选择加法器在考虑时序和硬件时延的情况下功能正常。

## 6.3 32 位流水线加法器

### 6.3.1 设计原理

在数字系统中，如果完成某些复杂逻辑功能需要较长的时延，就会使系统很难运行在较高的工作频率上。流水线设计（pipeline design）是经常用于提高系统运行速度的一种有效方法，即在长时延的逻辑功能块中插入触发器，将复杂的逻辑操作分步进行，减少每个部分的操作，从而提高系统的运行速度。

图 5.7 所示为采用 4 级流水线技术的 32 位加法器的原理框图，采用了 5 级锁存、4 级 8 位加法，整个加法只受 8 位加法器工作速度了限制，从而提高了整个加法器工作速度。

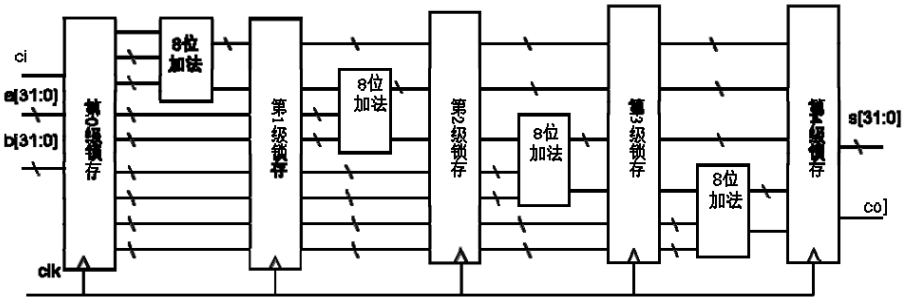



图 5.7　采用 4 级流水线技术的 32 位加法器原理框图

Figure 6: 采用 4 级流水线技术的 32 位加法器原理框图

### 6.3.2 Verilog HDL 代码与设计分析

1. **模块接口：**

   - clk(clock): 时钟信号，用于同步流水线各级的数据传输。
   - a,b:32 位输入信号，表示两个需要相加的 32 位二进制数。
   - ci(carry in): 进位输入，为低位加法器的进位。
   - s(sum):32 位的和输出（寄存器类型），表示加法结果。
   - co(carry out): 进位输出（寄存器类型），表示最高位的进位。

2. **内部信号：**

   - a_reg1, b_reg1, ci_reg1: 第 1 级流水线寄存器，用于锁存输入数据。
   - a_reg2[23:0], b_reg2[23:0]: 第 2 级流水线寄存器，存储剩余高 24 位数据。
   - s0_reg2[7:0]: 第 2 级流水线寄存器，存储第 0 段（bit[7:0]）的和。
   - c1_reg2: 第 2 级流水线寄存器，存储第 0 段产生的进位。
   - a_reg3[15:0], b_reg3[15:0]: 第 3 级流水线寄存器，存储剩余高 16 位数据。
   - s0_reg3, s1_reg3: 第 3 级流水线寄存器，存储第 0、1 段的和。
   - c2_reg3: 第 3 级流水线寄存器，存储第 1 段产生的进位。
   - a_reg4[7:0], b_reg4[7:0]: 第 4 级流水线寄存器，存储剩余高 8 位数据。
   - s0_reg4, s1_reg4, s2_reg4: 第 4 级流水线寄存器，存储第 0、1、2 段的和。
   - c3_reg4: 第 4 级流水线寄存器，存储第 2 段产生的进位。

3. **设计思路：**

   - 采用 5 级流水线结构，将 32 位加法分为 4 个 8 位加法段。

- 第 1 级：锁存输入数据 a、b、ci。
- 第 2 级：计算 bit[7:0] 的加法结果，同时将高 24 位数据传递到下一级。
- 第 3 级：计算 bit[15:8] 的加法结果，同时将高 16 位数据和 bit[7:0] 的结果传递到下一级。
- 第 4 级：计算 bit[23:16] 的加法结果，同时将高 8 位数据和 bit[15:0] 的结果传递到下一级。
- 第 5 级：计算 bit[31:24] 的加法结果，并组装完整的 32 位和与进位输出。
- 每个时钟周期可以接收一组新的输入数据，虽然单个数据的延迟为 5 个时钟周期，但吞吐量达到每周期一个结果（流水线填满后）。

Listing 3: 32 位流水线加法器 Verilog HDL 代码

```
`timescale 1ns/10ps
module pipeline_adder (
    input clk,
    input [31:0] a,
    input [31:0] b,
    input ci,
    output reg [31:0] s,
    output reg co
);

    // 第1级流水线寄存器
    reg [31:0] a_reg1, b_reg1;
    reg        ci_reg1;

    // 第2级流水线寄存器
    reg [23:0] a_reg2, b_reg2; // 剩余高24位数据
    reg [7:0] s0_reg2;         // 第0段(bit[7:0])的和
    reg       c1_reg2;         // 第0段产生的进位

    // 第3级流水线寄存器
    reg [15:0] a_reg3, b_reg3; // 剩余高16位数据
    reg [7:0] s0_reg3, s1_reg3; // 第0、1段的和
    reg       c2_reg3;         // 第1段产生的进位

    // 第4级流水线寄存器
    reg [7:0] a_reg4, b_reg4; // 剩余高8位数据
```

```
27      reg [7:0] s0_reg4, s1_reg4, s2_reg4; // 第0、1、2段的和
28      reg       c3_reg4;        // 第2段产生的进位
29
30      always @(posedge clk) begin
31          // --- 第1级流水线：输入数据寄存 ---
32          a_reg1 <= a;
33          b_reg1 <= b;
34          ci_reg1 <= ci;
35
36          // --- 第2级流水线：计算bit[7:0]，传递高24位 ---
37          {c1_reg2, s0_reg2} <= a_reg1[7:0] + b_reg1[7:0] + ci_reg1;
38          a_reg2 <= a_reg1[31:8];
39          b_reg2 <= b_reg1[31:8];
40
41          // --- 第3级流水线：计算bit[15:8]，传递高16位和bit[7:0]结果 ---
42          {c2_reg3, s1_reg3} <= a_reg2[7:0] + b_reg2[7:0] + c1_reg2;
43          a_reg3 <= a_reg2[23:8];
44          b_reg3 <= b_reg2[23:8];
45          s0_reg3 <= s0_reg2;
46
47          // --- 第4级流水线：计算bit[23:16]，传递高8位和bit[15:0]结果 ---
48          {c3_reg4, s2_reg4} <= a_reg3[7:0] + b_reg3[7:0] + c2_reg3;
49          a_reg4 <= a_reg3[15:8];
50          b_reg4 <= b_reg3[15:8];
51          s1_reg4 <= s1_reg3;
52          s0_reg4 <= s0_reg3;
53
54          // --- 第5级流水线：计算bit[31:24]，组装最终结果 ---
55          {co, s[31:24]} <= a_reg4 + b_reg4 + c3_reg4;
56          s[23:16] <= s2_reg4;
57          s[15:8] <= s1_reg4;
58          s[7:0]  <= s0_reg4;
59      end
60
61  endmodule
```

### 6.3.3 ModelSim 仿真与结果分析

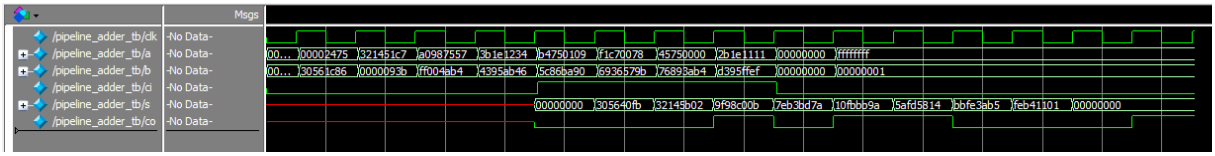



Figure 7: 32 位流水线加法器 ModelSim 仿真结果

【实验结果】:ModelSim 仿真结果表示，设计的 32 位流水线加法器功能正常，在第一个数据到达后，经过四个时钟周期后，输出了正确的结果。并实现了流水线的效果。

### 6.3.4 时序仿真与结果分析

在 vivado 中进行综合和实现后，对 32 位流水线加法器进行时序仿真，仿真结果如下，可以看到时序仿真中信号存在建立时间和时钟上升沿后的时延等现象

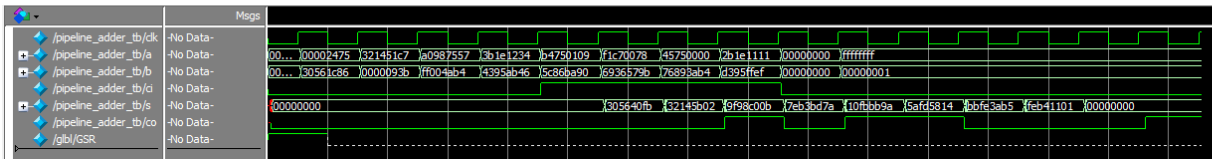



Figure 8: 32 位流水线加法器时序仿真结果

【实验结果】: 时序仿真结果表示，设计的 32 位流水线加法器在考虑时序和硬件时延的情况下功能正常

# 7 遇到的问题及解决方法

在设计 32 位流水线加法器时，一开始的仿真结果如下，最开始的 a 与 b 均为 0 的结果未能正确输出

Listing 4: pipeline_adder_tb.v

```
1    a=32'h00000000; b=32'h00000000;ci=0;
```

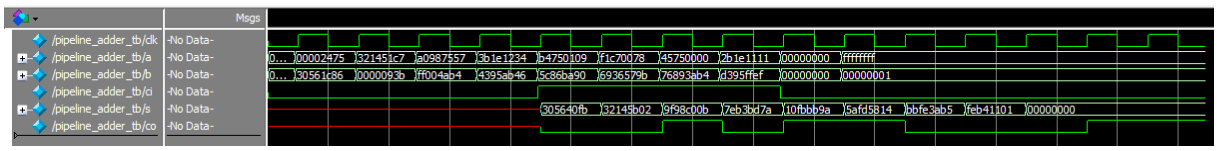



Figure 9: 32 位流水线加法器 ModelSim 仿真结果

后来发现，这是由于在第一个时钟上升沿（50ns）到来之前，a 和 b 的值就已经在 45ns 时刻被修改了。因此，流水线根本没有机会采到初始的 0 值。它在第一个周期锁存的就是 32'h00002475 和 32'h30561c86。所以，在波形上永远看不到 0+0 的计算结果 00000000

Listing 5: pipeline_adder_tb.v

```
1  initial begin clk=0; forever #(DELY/2) clk=~clk ;end
2  initial begin
3        a=32'h00000000; b=32'h00000000;ci=0;
4  #45   a=32'h00002475; b=32'h30561c86;ci=0;
```

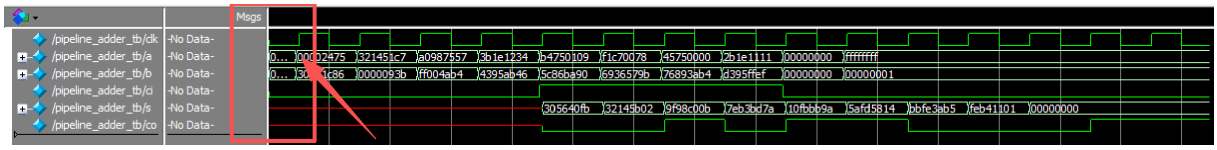



Figure 10: 32 位流水线加法器 ModelSim 仿真结果

基于此，将 45ns 改至 50ns 之后的 55ns，流水线就能采到初始的 0 值。在波形上就能看到 0+0 的计算结果 00000000

Listing 6: pipeline_adder_tb.v

```
1  initial begin clk=0; forever #(DELY/2) clk=~clk ;end
2  initial begin
3        a=32'h00000000; b=32'h00000000;ci=0;
4  #55   a=32'h00002475; b=32'h30561c86;ci=0;
```

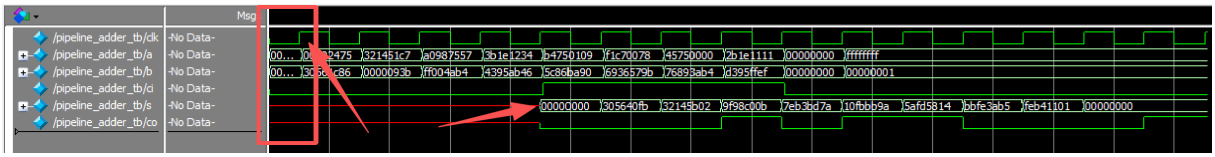



Figure 11: 32 位流水线加法器 ModelSim 仿真结果

同时，意识到，可能在原本的 tb 代码中，0 值只是为了初始化 a 与 b,45ns 时刻修改 a 和 b 的值就是为了让流水线在第一个周期就能锁存到非 0 的值，

# 8 思考题

1. 为什么要进行时序仿真？

**答：**时序仿真在功能仿真的基础上，考虑了实际电路中的**延迟**因素，能够更真实地反映电路的实际工作情况。

- **验证时序约束：**检验设计是否满足建立时间（setup time）和保持时间（hold time）等时序要求，确保电路在目标时钟频率下能够正常工作。
- **发现竞争冒险：**功能仿真假设门电路无延迟，无法发现由于延迟差异导致的竞争冒险（race condition）和毛刺（glitch）等问题。
- **评估性能指标：**通过时序仿真可以准确测量电路的最大工作频率、关键路径延迟等性能参数，为优化设计提供依据。

2. 采用流水线技术有什么优缺点？

**答：**

**优点：**

- **提高系统工作频率：**将长延迟的组合逻辑分割成多个较短的阶段，每个阶段的延迟减小，可以使用更高的时钟频率。
- **提高吞吐量：**虽然单个数据的处理时间可能增加（由于寄存器延迟），但多个数据可以同时在不同流水线阶段处理，整体吞吐量大幅提升。
- **提高资源利用率：**各个流水线阶段可以并行工作，充分利用硬件资源，避免资源闲置。
- **降低功耗：**在相同吞吐量下，流水线设计可以使用较低的工作电压，从而降低功耗。

**缺点：**

- **增加硬件开销：**需要在各级之间插入寄存器（触发器），增加了硬件资源消耗和芯片面积。
- **增加延迟（Latency）：**虽然吞吐量提高了，但单个数据从输入到输出的总时间增加，因为需要经过多个时钟周期。
- **流水线冒险：**存在数据冒险、控制冒险等问题，可能需要额外的硬件（如前递单元、冲突检测）或软件（如插入空泡）来解决。
- **负载不均衡：**如果各级流水线的延迟差异较大，整体性能将受限于最慢的一级，导致其他级资源浪费。
- **初始化和填充开销：**流水线启动时需要若干周期才能填满，在处理少量数据时优势不明显。